The Inspect Right as Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern

Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
1 May 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the three rights named in the source paper's "human-governed" commitment — the **inspect right** — as a standalone architectural commitment with independent operational content, separable from the modify and override rights with which it composes.

Abstract

The Coordination Knowledge Substrate (CKS) pattern's "human-governed" commitment names three rights — to inspect, to modify, and to override substrate content and orchestration rules — as jointly necessary for the architectural property the term carries. A separate note formalizes that joint commitment. This note formalizes one of the three rights, the inspect right, as having independent operational content that can be defended, implemented, and tested independently of the other two. The motivation is concrete: oversight roles such as auditors, compliance reviewers, regulatory inspectors, and security reviewers exercise primarily the inspect right without exercising modify or override authority, and their roles must remain architecturally describable as CKS-coherent governance. The note states what the inspect right requires of the host environment and the substrate's design, distinguishes it from three adjacent commitments commonly conflated with it (audit logging, transparency, explainability), enumerates the failure modes that violate it specifically, and provides an operational test for whether a given system implements the inspect right separable from the broader human-governed commitment.

1. Why the inspect right needs to be formalized as standalone

The CKS pattern's "human-governed" commitment names three rights together: to inspect, to modify, and to override substrate content and orchestration rules at any time during the substrate's existence (§2.1, §3.3 of the source paper). A separate derivation note formalizes that joint commitment as an authority architecture rather than a labor or review commitment.

The joint framing is correct as far as it goes, and this note does not contradict it. But it leaves a class of cases architecturally underspecified. A regulated organization typically separates oversight from operational authority by design: an internal auditor reads but does not write; a compliance reviewer

reads decisions but does not change them; a regulatory inspector observes but is not a party to operational decisions; a security reviewer assesses configuration without modifying it. Each of these roles exercises authority over the substrate, but the authority each exercises is overwhelmingly the inspect right, not the modify or override rights. If the human-governed commitment is treated as a single composite — three rights as one unitary architectural property — these roles look like "partial governance," and the architecture has no principled vocabulary for what they exercise. The temptation is then to implement them as workflow features (read-only views generated by the application layer, audit dashboards backed by snapshot exports, regulator portals over derived projections), all of which sit at a different architectural layer than the right the source paper commits to.

The remedy is to name the inspect right as a standalone architectural commitment with independent operational content. A role granted only the inspect right — within whatever scope the deployment authorizes — is exercising a real architectural authority, not a derived or simulated one. Naming this content explicitly is what makes inspect-only roles describable as CKS-coherent governance and prevents the slide into workflow-feature substitutes.

2. The inspect right, defined precisely

In the CKS pattern, a human exercises the **inspect right** when they read substrate content or orchestration-rule content directly, without LLM intermediation as a precondition, in inspectable form, at the time of their choosing. The right has four operational components.

(a) Read access to substrate content within authorized scope. A human with appropriate access can read any substrate content the deployment authorizes them to see — entities, relationships, decisions, rationale, conflicts, provenance — as the substrate actually carries it. Authorization scope is determined by the broader human-governed authority architecture; the inspect right operates within whatever scope is authorized.

(b) Read access to orchestration rules within authorized scope. A human with appropriate access can read any orchestration rule that governs cell-level behavior, including the rule's authorship, version history, and current state. Orchestration rules are substrate-level content (§2.1); the right extends to them by the same logic that grounds (a).

(c) Direct access without LLM intermediation as a precondition. The human may use LLM-assisted tooling on top of the substrate — search, summarization, query reformulation, semantic navigation — and such tooling is often valuable. But the human must also be able to read the underlying substrate content directly, in inspectable form, when they choose. The LLM is permissible as an adjacent tool; it cannot be the gate.

(d) At-the-time-of-choosing access without scheduling or approval gating. The right is exercisable when the human decides to exercise it, not when a workflow permits it. Scheduled review windows, approval-gated sessions, and time-locked inspection portals all fail the temporal component, even when they grant ample read access otherwise. This component is what §3.3 of the source paper carries when it commits the human-governed commitment to authority "at any time."

The four components together define what the inspect right requires of the host environment and the substrate's representational form. Failing any one — even with the other three robustly satisfied — fails the inspect right architecturally.

3. What the inspect right does NOT require

The standalone treatment of the inspect right is not a maximalist treatment. Stating precisely what the right does not require is what keeps the standalone framing from drifting into something stronger than the source paper supports.

It does not require modify access. A reader who can inspect substrate content but cannot change it is exercising the inspect right at full architectural scope. Modify access is a separate right; whether a given role holds it is a deployment decision that does not affect the inspect right's content.

It does not require override authority. A reader who can inspect a cell's decision and disagree with it but cannot override the decision is exercising the inspect right, not a degraded form of it. Override is the third of the three rights; like modify, it is separable.

It does not require comprehension. The inspect right is access, not understanding. A reader who can read substrate content but does not understand its domain is still exercising the right architecturally; whether they comprehend what they read is outside the architecture's scope.

It does not require LLM-assisted summary or explanation. Adjacent tooling may provide summaries, explanations, or generated narratives over substrate content, and such tooling is often useful. The inspect right is satisfied by direct access alone.

It does not require continuous monitoring. The right is exercisable when the reader chooses; it imposes no obligation to inspect on any schedule, at any frequency, or in response to any event. A reader who exercises the right once and never again has exercised it as fully, architecturally, as one who reads continuously.

4. What the inspect right is NOT

Three adjacent commitments are commonly conflated with the inspect right. Each is a real and reasonable commitment in some other architecture; naming what the inspect right is not is what prevents the misreading.

Not audit logging. Audit logging records events external to the substrate — who accessed what, when, from where, through which interface — for after-the-fact review. The inspect right is access to substrate content itself, not access to logs about access. The two operate at different layers and serve different purposes. A system can have complete audit logging while failing the inspect right entirely, if the logged-about substrate is itself unreadable in inspectable form. Audit logging is neither necessary for the inspect right nor sufficient for it.

Not transparency commitments. Transparency in AI systems typically names a commitment that the system provides explanations of its decisions in human-understandable form. The inspect right is access

to substrate content as it actually exists, not access to explanations generated about substrate content. Transparency commitments may be served by the inspect right (when the substrate carries the rationale for decisions) or by adjacent mechanisms (generated explanations, summaries, rationale strings). The two are different commitments operating at different scopes.

Not explainability. Explainability commitments aim to make AI decisions reconstructable by humans, often through generated post-hoc rationales. The inspect right gives access to whatever rationale the substrate carries — which, in CKS, is substantial when the deployment captures decision rationale, authorship, and conflict provenance as substrate content (§3.1, §11.3). But the inspect right does not commit to the substrate carrying any particular form of explanation. Whether the substrate contains rationale depends on what the deployment chooses to capture; the inspect right's commitment is to make whatever the substrate carries readable, not to specify what it must carry.

The three distinctions together are what keep the inspect right's content precise: the right is access to substrate content directly, not the commitment that the access is logged, that explanations are generated, or that decisions are reconstructable on demand.

5. What the inspect right makes possible at the architectural layer

Naming the inspect right as standalone architectural commitment has four consequences at the architectural layer.

Inspect-only governance roles become describable. Auditors, compliance reviewers, regulatory inspectors, security reviewers, and other oversight roles can be granted inspect access within authorized scope, without modify or override authority, and the architecture treats their exercise of authority as CKS-coherent governance. This gives downstream implementations a principled vocabulary for separation-of-duties configurations that many regulated settings already require.

Non-specialist inspection follows from tool-agnosticism. The source paper's tool-agnosticism commitment (§7.1) names three minimal requirements for any environment that instantiates the pattern: persistent structured state, human read/write access, and LLM access to substrate content. The second of these grounds the inspect right's direct-access component. Because the right is satisfied in any environment meeting that requirement, it is exercisable in commodity tools by humans without specialist training. A non-specialist auditor reading a spreadsheet substrate is exercising the inspect right at full architectural scope (§7.4).

Independent verification becomes possible. A human exercising the inspect right can verify the system's claims about its state — what was decided, by whom, under what authority, with what rationale, what conflicts remain — by reading the substrate directly, without depending on the system's self-reports generated through the LLM mediator. The substrate is the source of truth (§11.3); the inspect right is what makes that source-of-truth status verifiable in practice rather than only in principle.

The substrate's deterministic state behavior becomes observable. The source paper commits the substrate to deterministic state behavior at the coordination layer — substrate content, once written under appropriate authority, is what the substrate carries until written again under appropriate authority

(§4.1, §11.3). The inspect right is what makes that determinism observable. Without it, the determinism guarantees are unverifiable in practice, regardless of how robustly the architecture commits to them in principle.

These four are not new commitments; they follow from treating the right as having independent operational content.

6. Failure modes that violate the inspect right

A system can fail the inspect right specifically, even when it satisfies the modify and override rights and broader governance commitments. Five failure modes name the most common ways this happens.

(a) LLM-mediated inspection. When the only path to substrate content runs through an LLM call — the human "asks" the system what the substrate contains, and the LLM produces a summary — the right is violated. The LLM may be a useful adjacent tool; it cannot be the gate.

(b) Embedded-only access. When substrate content is held in a representation only readable through embedding queries or vector search, and direct human reading produces only opaque numerical artifacts, the right is violated. Embedding indexes are a permissible adjacent representation; they are not a permissible primary one for inspection purposes.

(c) Scheduled-window inspection. When inspection is permitted only during scheduled review windows, audits at predetermined intervals, or approval-gated sessions, the at-the-time-of-choosing component is violated.

(d) Vendor-revocable inspection. When the host environment, vendor, or runtime middleware can in principle prevent a human from reading authorized substrate content, the right is architecturally compromised, regardless of how rarely the prevention occurs in practice. The architectural commitment is to a property of the system's design, not to a vendor's current policy. Inspectability cannot be revocable as a system feature.

(e) Compiled-only state. When substrate content exists only in a compiled, optimized, or otherwise transformed form that requires reverse-engineering to read — a graph collapsed into vectors, a structured document minified beyond human reading, an archive encoded in a vendor-specific binary — the right is violated. The substrate must remain in a form humans can read directly, not only after reconstruction.

A system that exhibits any of (a)–(e) does not implement the inspect right specifically, and naming the failure precisely is what allows downstream remediation.

7. Operational test

A system implements the inspect right specifically if and only if all of the following are true at all times during the substrate's existence:

1. A human with appropriate access can read any substrate content within authorized scope, in inspectable form, without LLM intermediation as a precondition.

2. A human with appropriate access can read any orchestration rule within authorized scope, including its authorship, version history, and current state.
3. The inspection action does not require scheduling, approval, or workflow gating; it is exercisable at the human's chosen time.
4. No LLM operation, vendor policy, or runtime middleware can in principle prevent (1), (2), or (3) for authorized humans.
5. Substrate content is held in a form readable by humans directly, not only through machine-mediated retrieval.

A system that fails any of (1)–(5) does not implement the inspect right specifically, even if it satisfies the modify and override rights and the broader human-governed commitment in some other respect. Such a system is not CKS-coherent on the inspection axis, and downstream work that relies on its inspection guarantees should be scoped accordingly.

8. Conclusion

Conflating the three rights into a single composite commitment makes inspect-only governance roles difficult to describe architecturally and tempts implementers to handle them as workflow features rather than as architectural properties. Failing the inspect right specifically while claiming to satisfy human-governed produces systems where modify and override authority is exercisable but inspection is gated — the inverse of what the commitment requires, since a modify or override action taken without the prior ability to read what is being changed is not governance in any robust sense.

Naming the inspect right as standalone architectural commitment gives downstream implementers a precise specification of what their inspection mechanism must satisfy, independent of how modify and override are handled. It makes inspect-only oversight roles describable as CKS-coherent governance rather than as partial or simulated authority. And it preserves the source paper's joint framing of human-governed: the standalone treatment names independent operational content within a composite right, not a fourth right alongside the three.

Subsequent work that implements, extends, or argues against the CKS inspection commitment should use "the inspect right" in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Inspect Right as Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern.* 1 May 2026. ORCID: 0009-0004-8065-3235.